Music Genre Classification

The problem

Classify a music track into its top-level genre from audio alone.

Dataset: Free Music Archive (FMA)

Feature representations compared:

- Hand-crafted: MFCC (Mel-Frequency Cepstral Coefficients)
- Pre-computed: 518-dimensional FMA descriptors
- Extracted: CLAP embeddings (deep audio-language model)

Models: Logistic Regression · Random Forest · SVM · LightGBM · CNN

Evaluation: macro F1 · macro AUROC · two split strategies

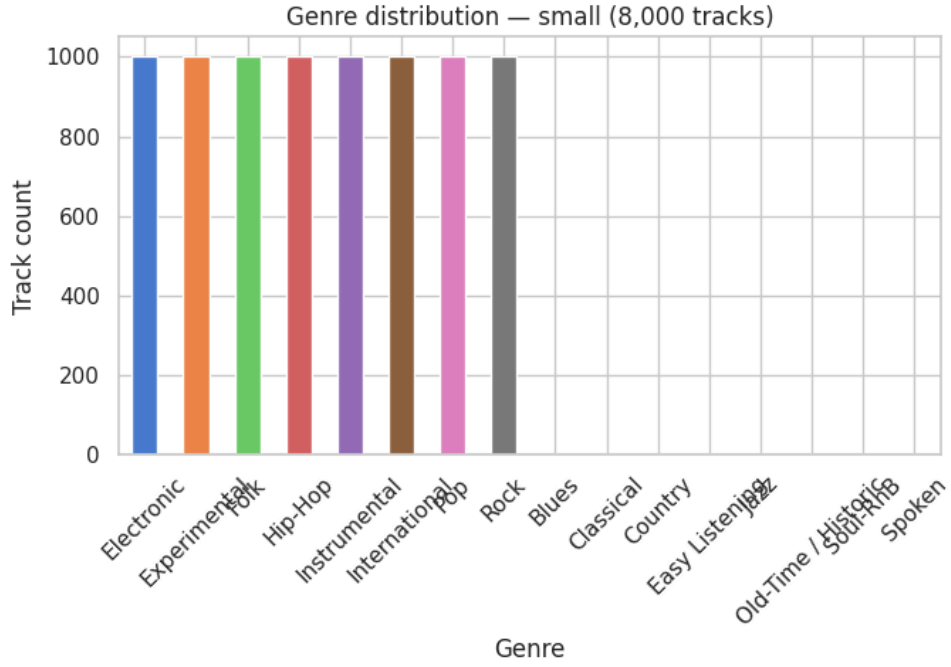
Dataset — Free Music Archive

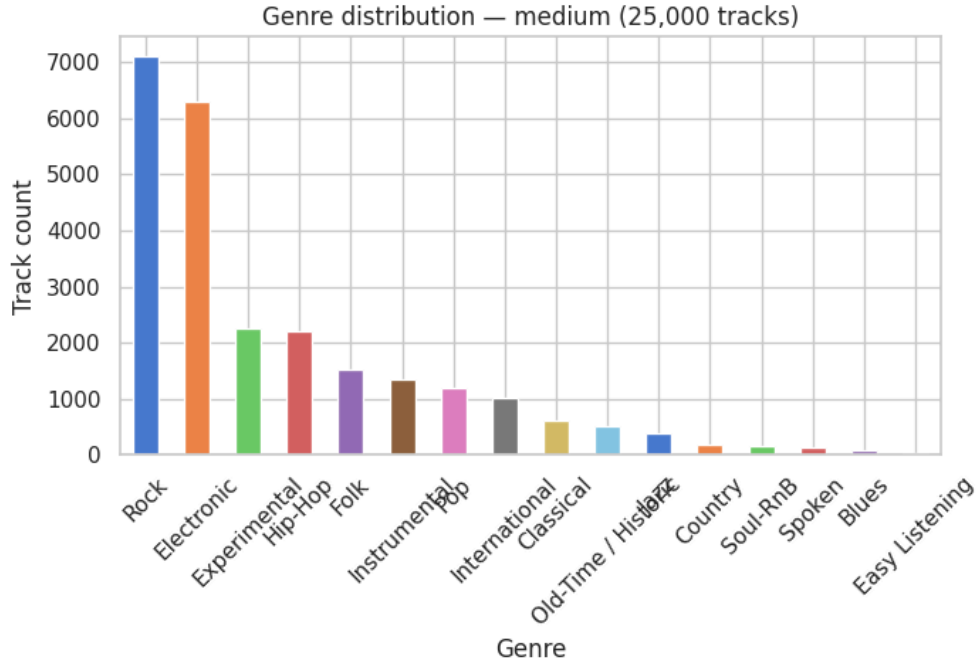
Two subsets, two very different problems:

Subset	Tracks	Genres	Class balance
fma_small	8,000	8	Balanced (1,000/genre)
fma_medium	25,000	16	Highly imbalanced

After cleaning (missing audio, removing *Instrumental* catch-all):

Subset	Clean tracks	Train	Test
small	6,997	5,597	1,400
medium	23,634	18,907	4,727





Small: perfectly balanced. Medium: Rock dominates, some genres have < 100 tracks.

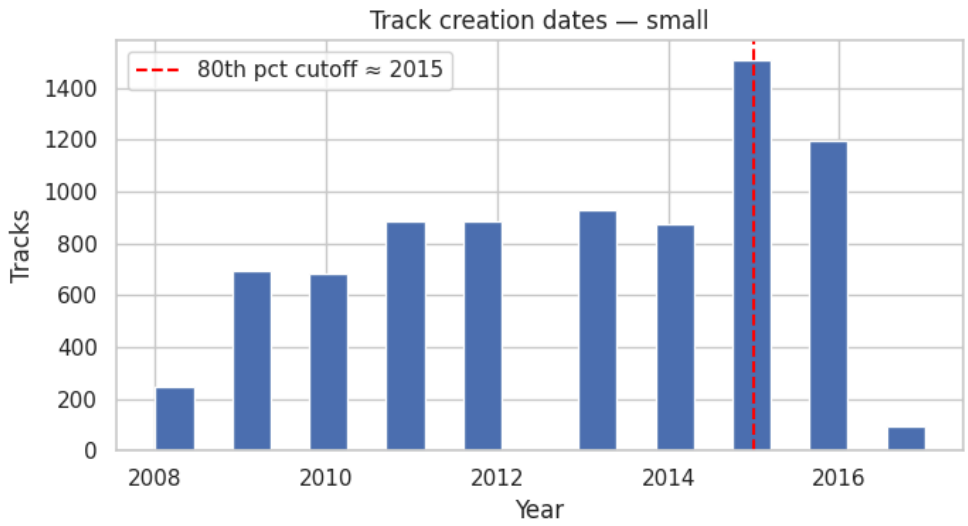
Holdout strategy: random vs time-based split

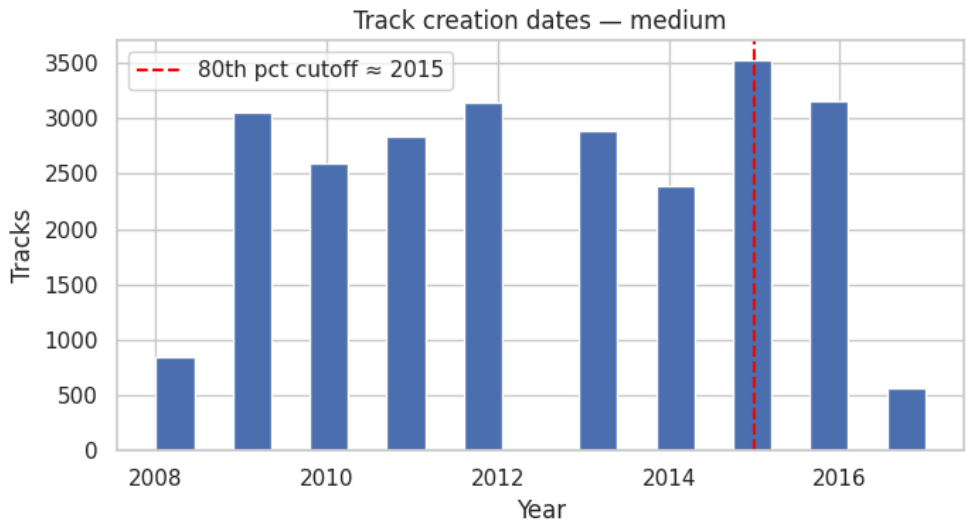
Random split — shuffle, 80/20

- Training and test tracks come from the same time window
- Inflates reported performance

Time-based split — train on older tracks, test on newer

- Mirrors real deployment: predictions always run on future music
- Cutoff: 80th percentile of upload date ≈ 2015





The red line marks the train/test boundary. Tracks uploaded after 2015 → test set only.

How to analyze audio?

A 30-second audio clip = ~660,000 raw pressure samples. Raw samples are not useful directly.

Three strategies:

1. **MFCC** — compute perceptually-motivated statistics from the waveform
2. **FMA pre-computed features** — 518 spectral/tonal descriptors provided by the dataset
3. **CLAP embeddings** — feed audio into a pre-trained deep model, get a 512d vector

Visual approach (CNN): convert audio to a 2D mel spectrogram image, train a convolutional network on it

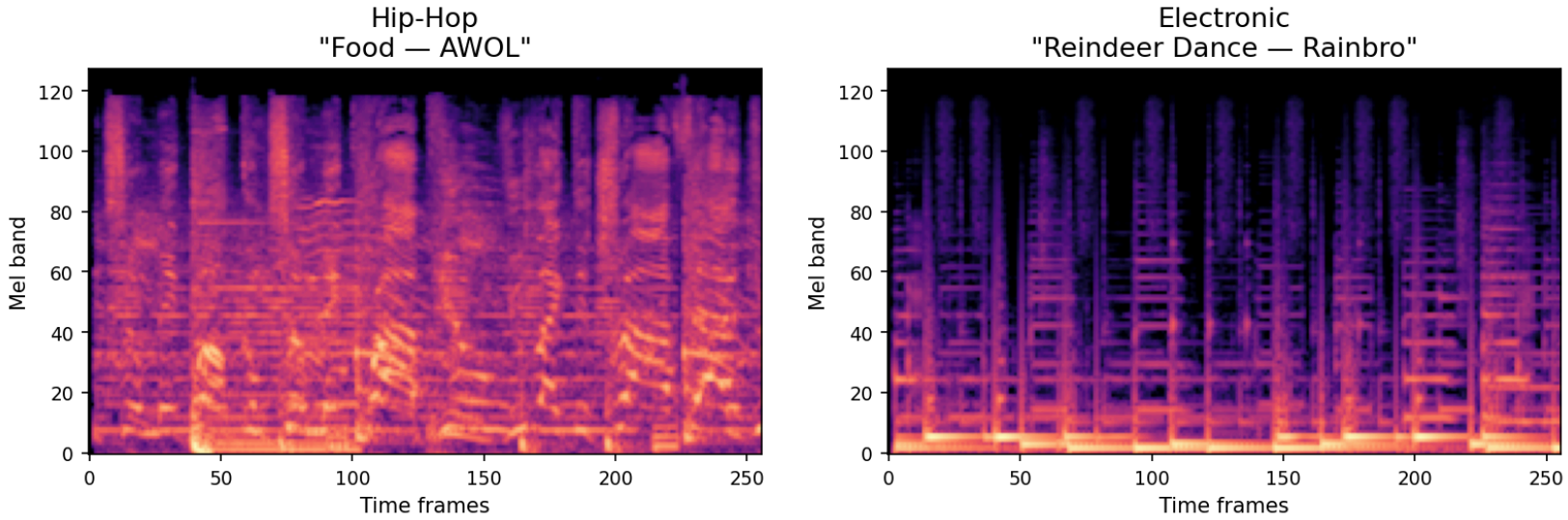
What is a spectrogram?

Fourier Transform converts time → frequency domain.

Mel Spectrogram — 2D map of frequency energy over time:

- x-axis: time (sliding windows ≈ 23ms each)
- y-axis: frequency on the **Mel scale** (mimics human pitch perception)
- color: log energy

The Mel scale spaces frequency logarithmically — equal spacing = equal perceived pitch interval.



What is MFCC?

Mel-Frequency Cepstral Coefficients — compact description of a sound's timbre.

Pipeline:

1. Compute Mel spectrogram (128 bands)
2. Log of each band's energy
3. DCT → decorrelates bands, packs information into first coefficients
4. Keep first **20 coefficients** (coarse timbre shape)

From 20 coefficients → 140 features:
compute over all frames, then summarise per clip with 7 statistics (mean, std, skew, kurtosis, median, min, max).

Parameters:

- `sr=22050` — sample rate (explained on next slide)
- `n_fft=2048` — window size for each Fourier Transform; larger = better frequency resolution, lower time resolution

What is CLAP?

Contrastive Language-Audio Pretraining — a deep audio embedding model.

Model: `laion/clap-htsat-unfused` — pre-trained on **630,000+ audio-text pairs**

Contrastive training: audio clip of hip-hop and text "hip-hop music with beats" → close vectors. Unrelated pairs → distant vectors.

How we use it:

- Feed audio → get a 512-dimensional vector
- librosa handles resampling to 48 kHz before passing audio to the model
- Computed once per track (GPU), saved to `clap_features_small.csv`
- Treat embeddings as features for classical ML

How we computed MFCC

```
import librosa, numpy as np, scipy.stats

def extract_mfcc(path, n_mfcc=20):
    y, sr = librosa.load(path, sr=22050)
    mfcc = librosa.feature.mfcc(
        y=y, sr=sr, n_mfcc=n_mfcc, n_fft=2048, hop_length=512
    )
    stats = []
    for coeff in mfcc:
        stats.extend([
            np.mean(coeff), np.std(coeff), np.median(coeff),
            np.min(coeff),  np.max(coeff),
            scipy.stats.skew(coeff), scipy.stats.kurtosis(coeff)
        ])
    return np.array(stats)   # shape: (140,)
```

`librosa.load` handles resampling automatically — audio files may be at any original rate; librosa resamples to `sr=22050` on load.

Data processing

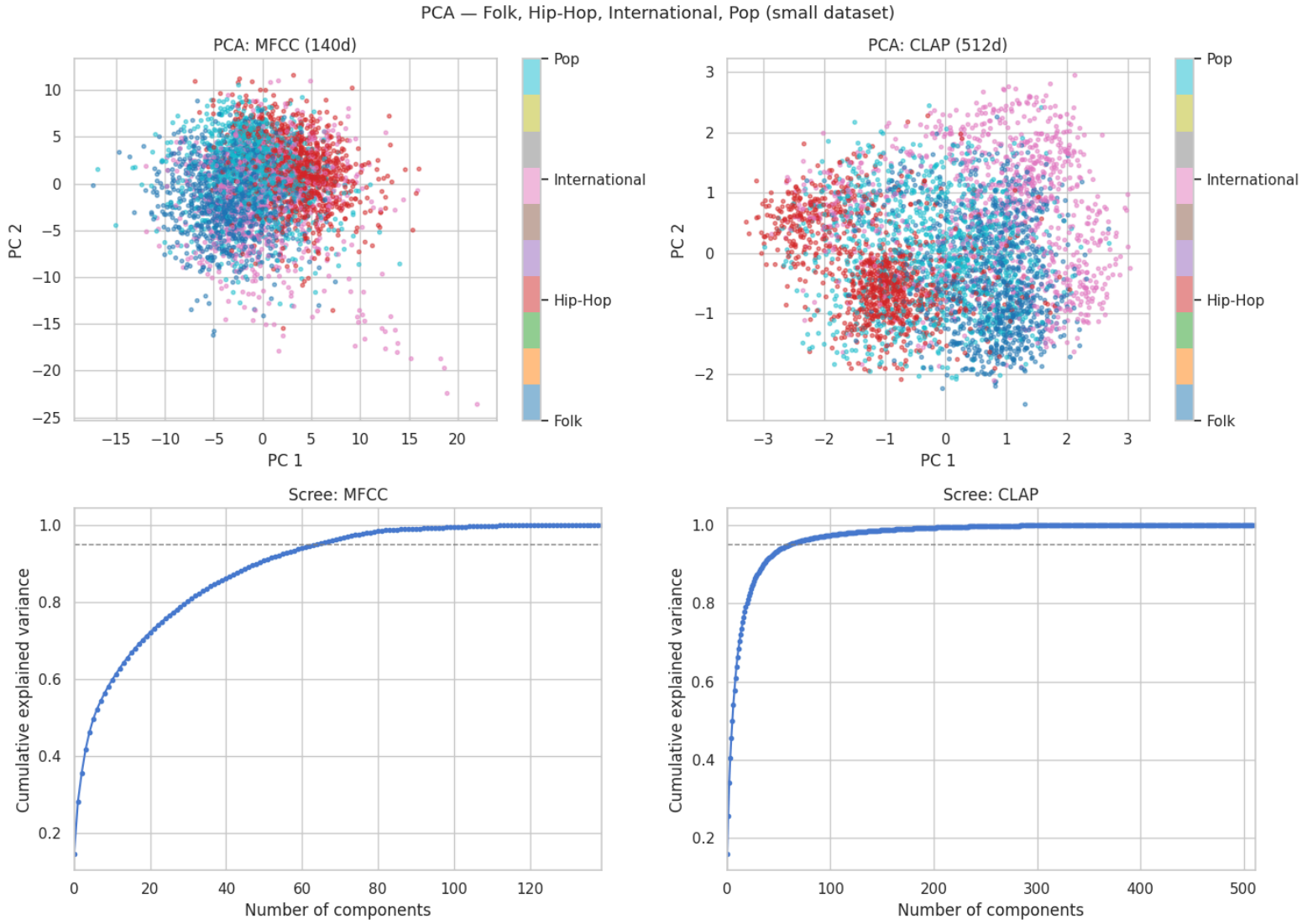
Cleaning:

- Drop tracks where librosa failed to load audio
- Remove *Instrumental* — a catch-all bin where tracks that didn't fit any real genre ended up; mixes spoken word, ambient noise, field recordings, and other uncategorised content
- No deduplication (FMA track IDs are unique)

Dimensionality reduction — PCA:

- All vectors (140d / 518d / 512d) → **100 components**
- PCA fitted on training split only — prevents leakage
- Scaler (StandardScaler) fit on train, applied to test

PCA — feature space visualisation



CLAP shows clearer genre separation in 2D. MFCC scree curve saturates faster.

Training — models

Model	Key setting	Tuned?
Logistic Regression	C (L2 regularisation)	Bayesian opt
Random Forest	n_estimators, max_depth, min_samples_split	Bayesian opt
SVM	RBF kernel, C	Bayesian opt
LightGBM	num_trees, lr, num_leaves	Bayesian opt
CNN	Adam · 20 epochs · Dropout	Fixed

All classical models receive **100-d PCA-reduced features**.

CNN input: `(1, 128, 256)` log-mel spectrogram per track.

Storage cost of pre-generating spectrograms:

- small dataset: ~5 GB of `.npy` files
- medium dataset: ~15 GB of `.npy` files

Bayesian Optimisation

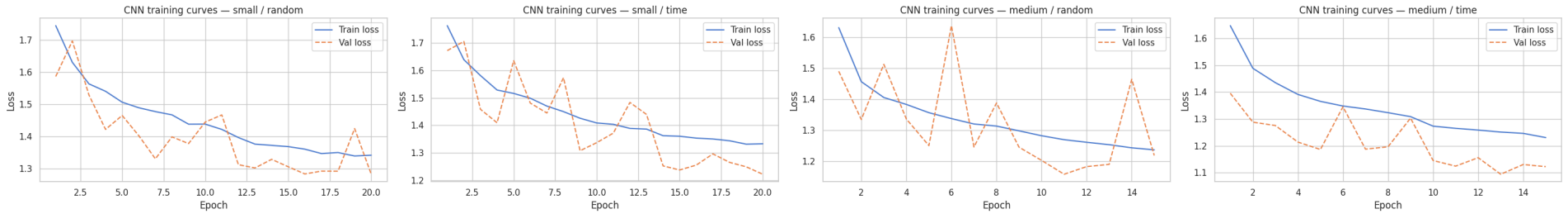
Grid search is exponential. Bayesian optimisation builds a probabilistic model of `hyperparams → metric`:

1. Evaluate a few random configurations
2. Fit a surrogate model (TPE) on observed results
3. Use acquisition function to pick the next most promising config
4. Evaluate, update surrogate, repeat

Setup — Optuna TPE:

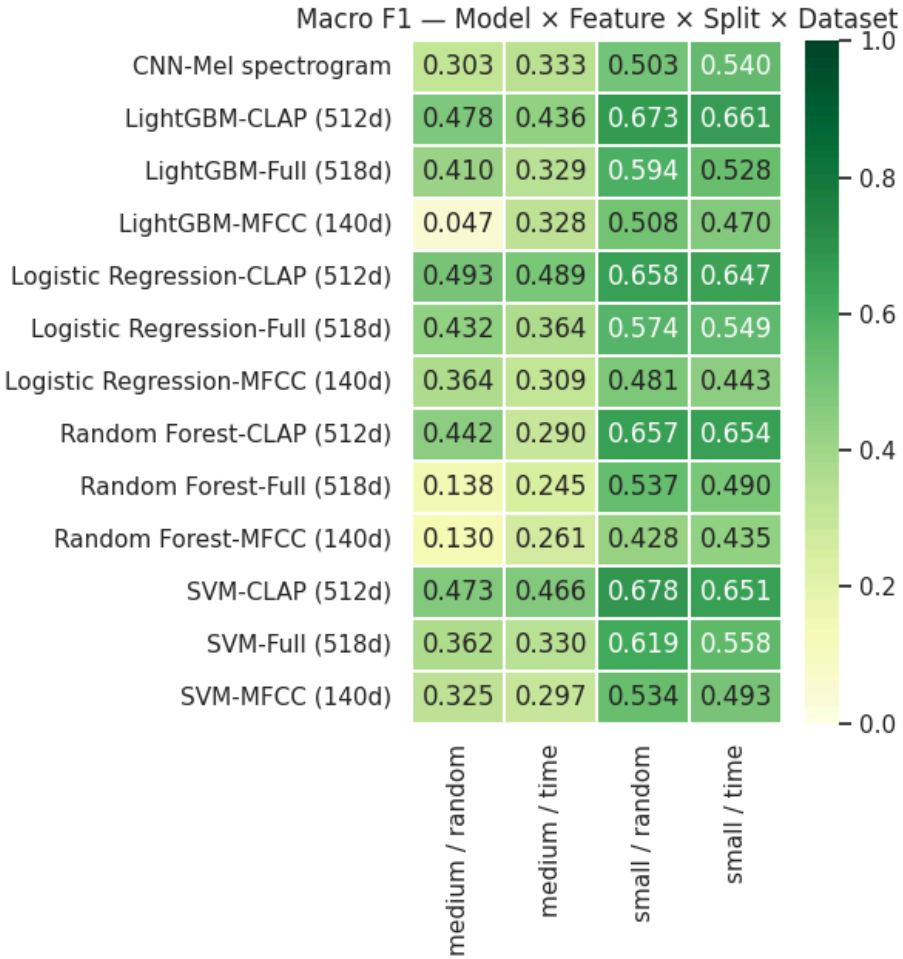
- 10 trials · 2-fold stratified CV · objective: macro AUROC OVR
- LR: C ∈ [0.01, 10]
- SVM: C ∈ [0.1, 10]
- LightGBM: num_trees ∈ [100, 300] · lr ∈ [0.05, 0.2] · num_leaves ∈ [31, 127]
- Random Forest: n_estimators ∈ [100, 500] · max_depth ∈ [5, 30] · min_samples_split ∈ [2, 10]

CNN training curves

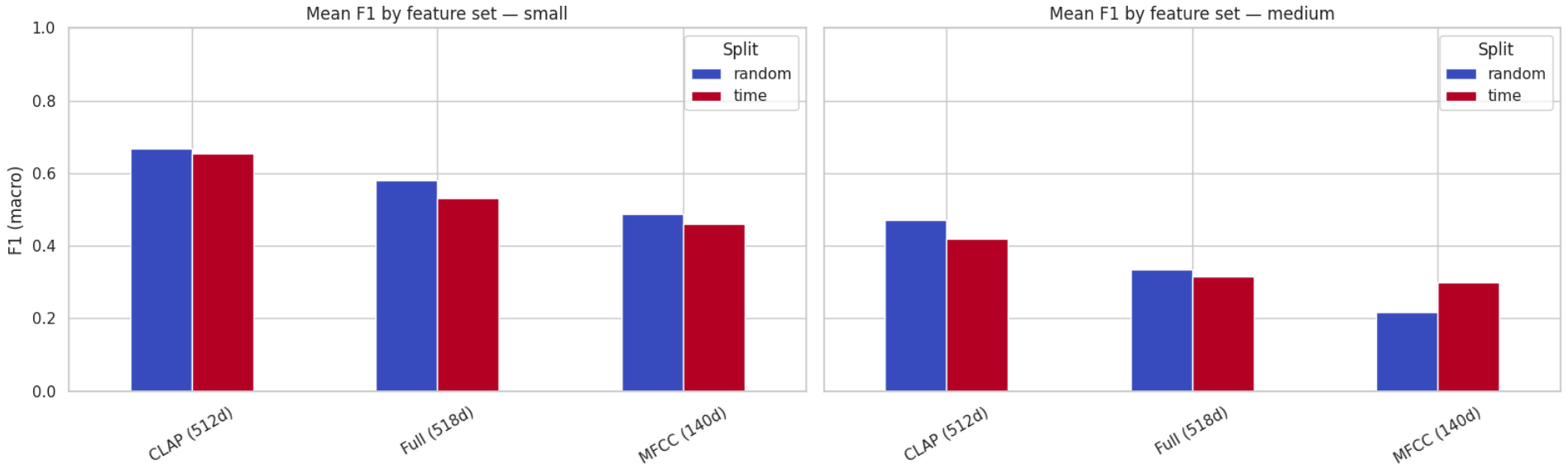


Four runs: small and medium × random and time split. On medium, high validation variance — 15 genres, imbalanced classes. Unlike classical models, the CNN improves slightly on the time split (+0.037 F1 on small).

Results — macro F1 heatmap

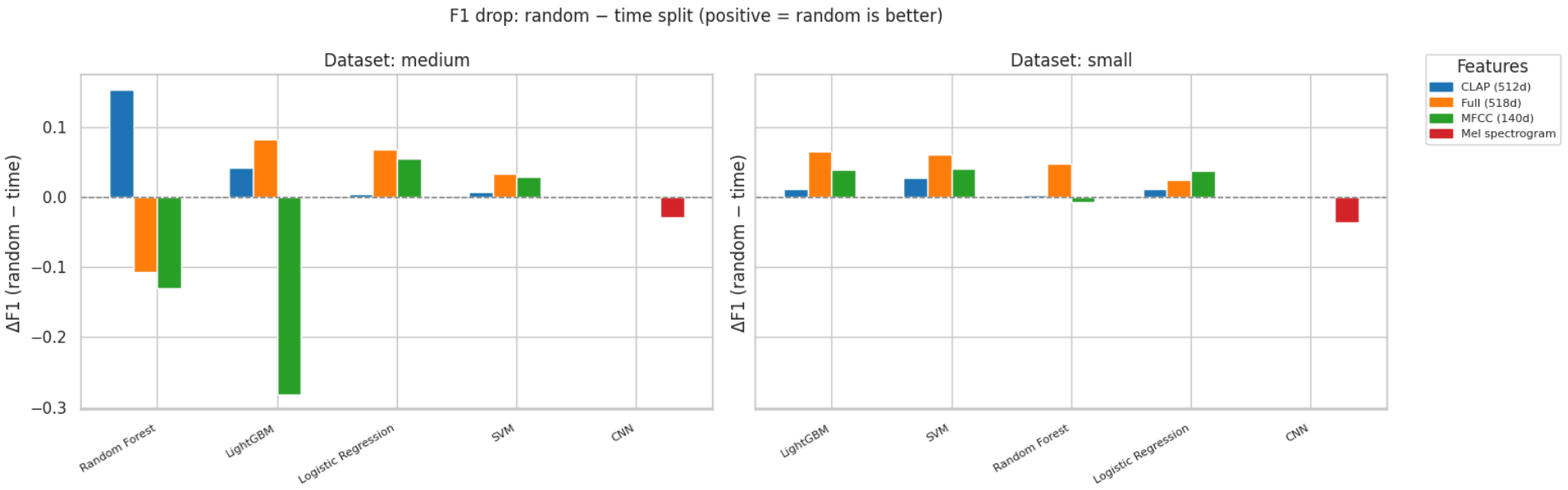


Results — F1 by feature set (random vs time split)



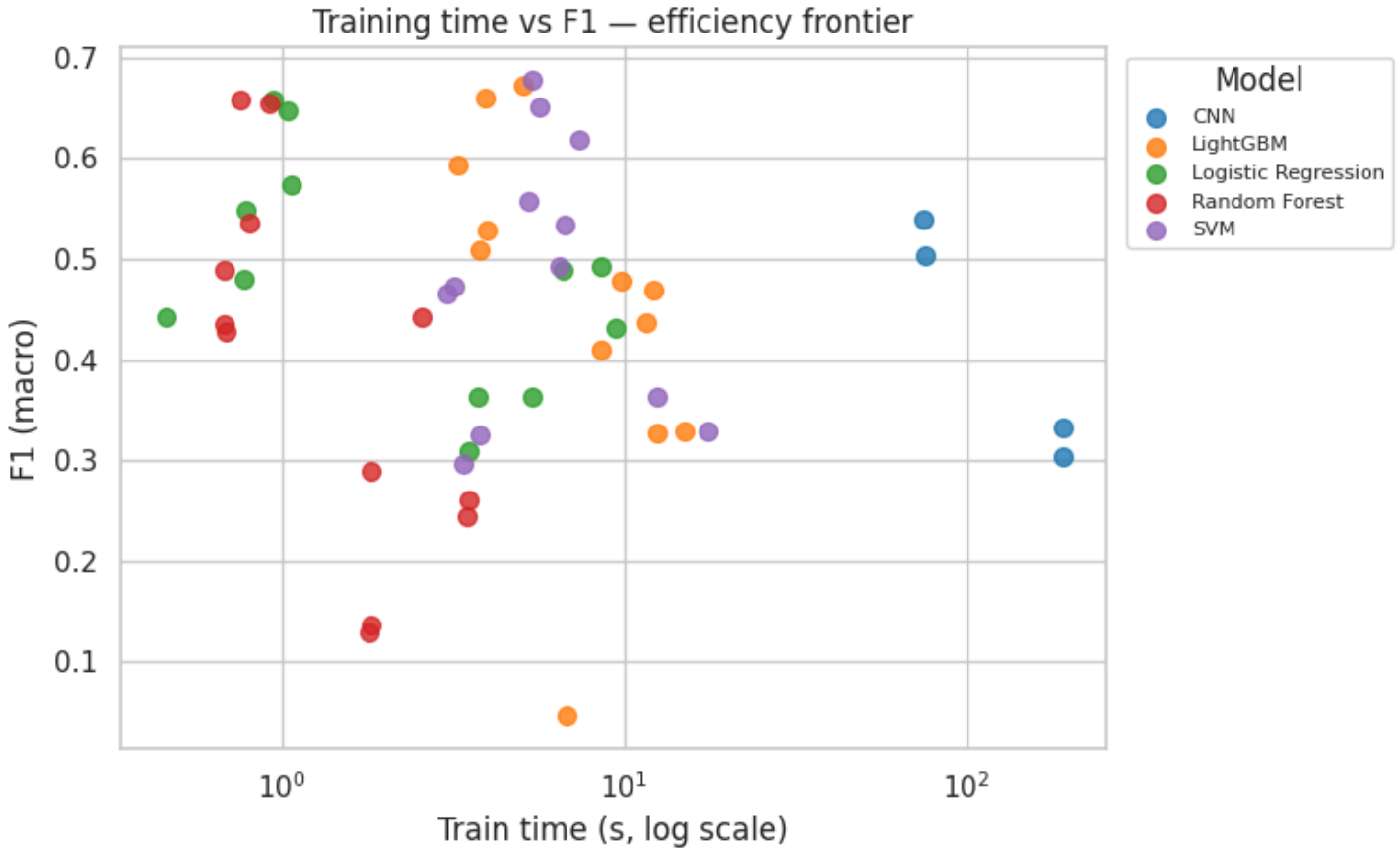
CLAP is consistently best. Time split (red) always below random (blue) — realistic performance is lower than papers typically report.

Results — F1 drop: random → time split



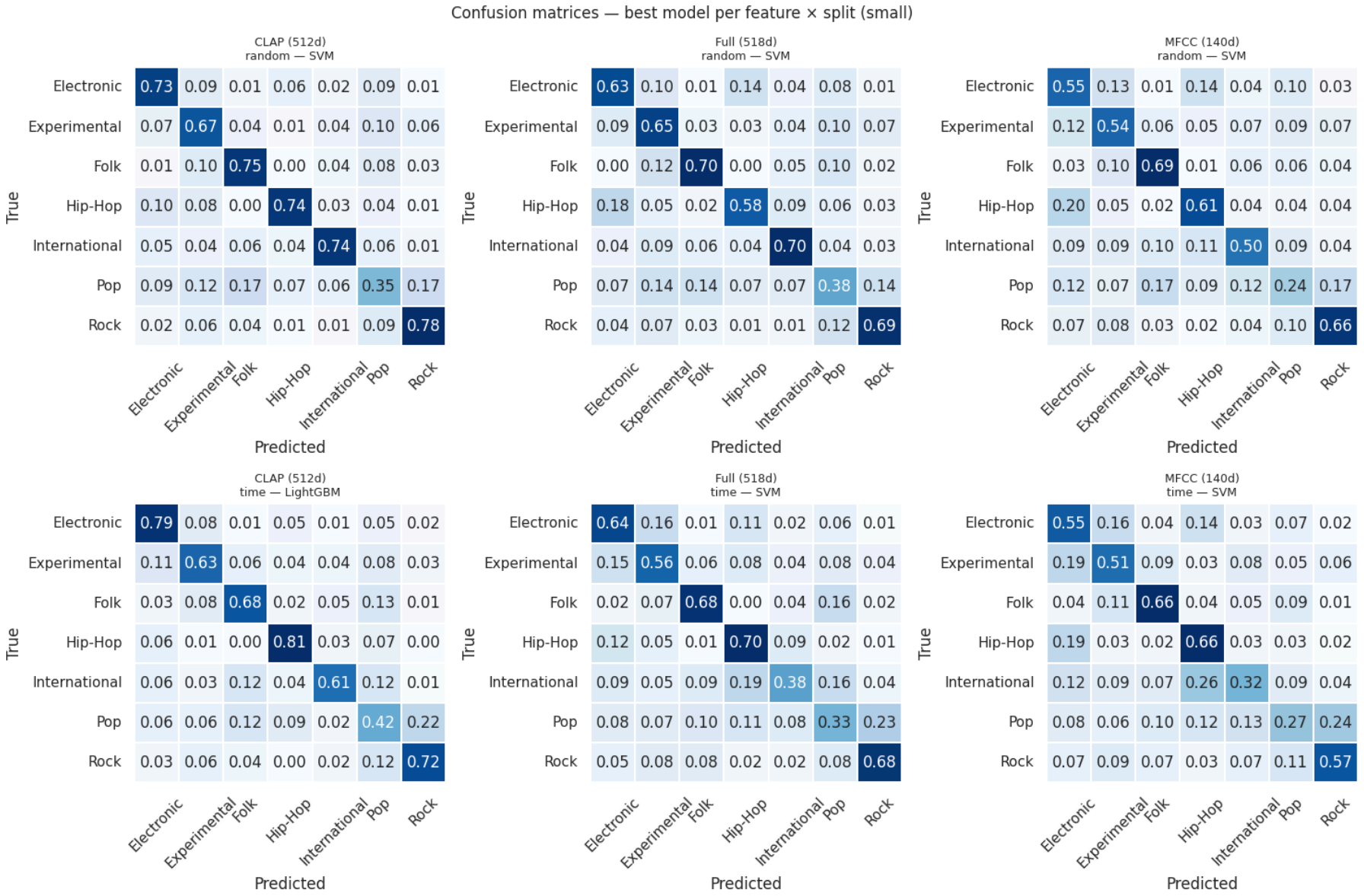
Classical models all degrade with time split (positive = random was better). CNN goes the opposite direction — time split actually helps it slightly. CLAP is the most robust classical option.

Results — training time vs F1

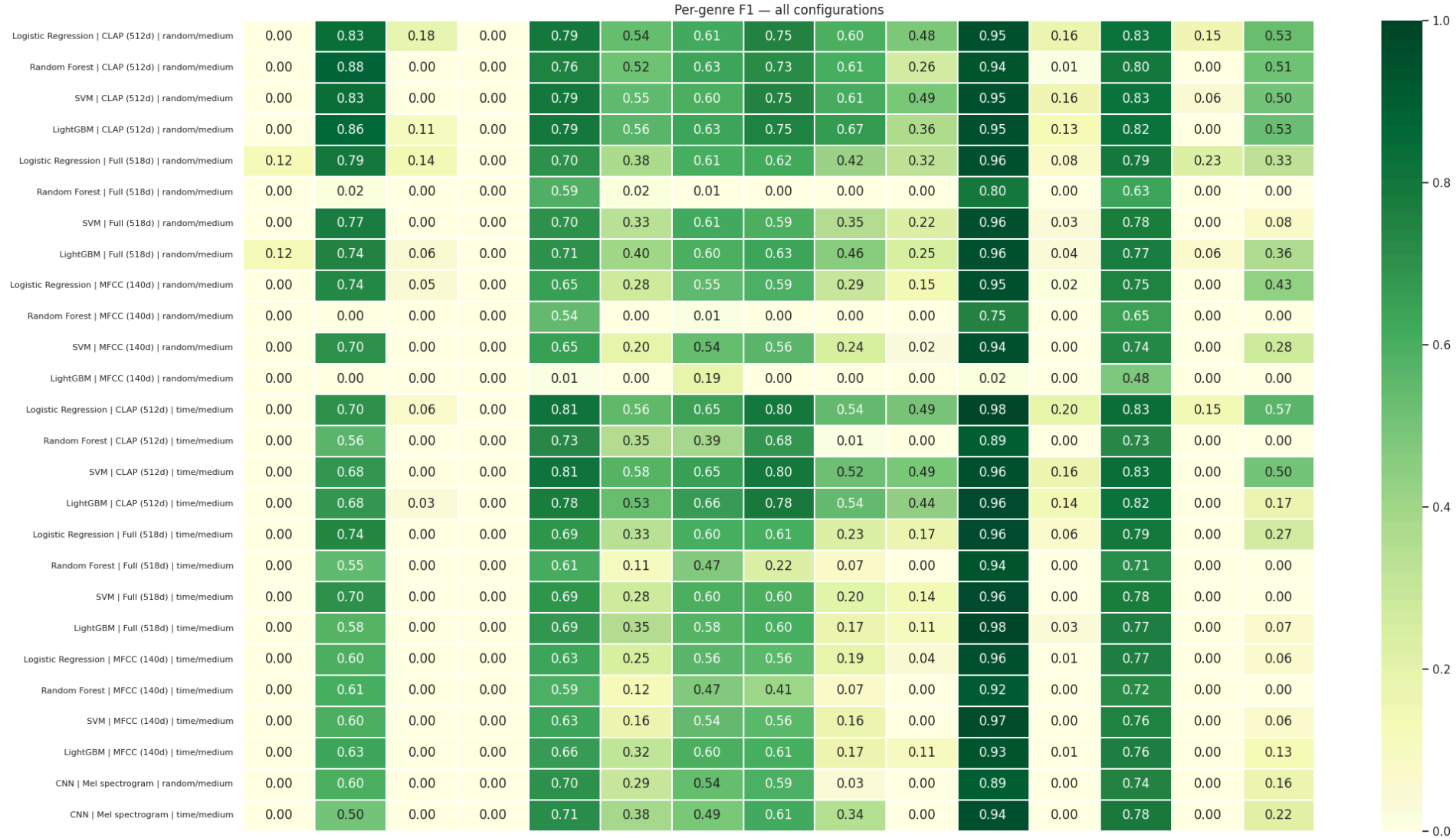


Classical ML with CLAP: top-left corner — highest F1, fastest training. CNN: bottom-right — lowest F1, slowest. No efficiency argument for the CNN at this dataset size.

Confusion matrices — best model per feature × split



Per-genre F1 — all configurations



Interpretation

CLAP wins — pre-training on 630k audio-text pairs already encodes genre. No domain engineering.

Classical ML beats CNN — 7k tracks is too small for a CNN to compete. SVM leads by 17 pp F1 at 13× lower training time.

Time split tells the real story — random split is the optimistic ceiling. Classical models drop 1–7 pp F1 on time split. The CNN is an exception: it picks up ~4 pp, possibly because raw spectrograms are less tied to the historical label distribution than pre-extracted statistics.

Pop is a label problem, not a model problem — boundary between independent pop, folk-pop, and rock is genuinely ambiguous. Hyperparameter tuning will not fix it.

Scaling to medium made things harder — 15 genres + severe imbalance drops F1 from 0.68 → 0.49. AUROC stays high (0.94)

Summary

Best result	SVM + CLAP · small · random · **F1 = 0.678 · AUROC = 0.925**
Production estimate	SVM + CLAP · small · time split · **F1 = 0.651 · AUROC = 0.910**
Classical vs CNN	**+17 pp F1 · 13× faster training**

Thank you

Kamil Kawula, Marek Małek